Reviewer Pack — Hilbert-Compression Floor on Property-A Gadgets Forces α·Q(f…

Entry 15542d600c45 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 23:06:08 UTC

Hilbert-Compression Floor on Property-A Gadgets Forces α·Q(f) Multiplicity in Protocol Pullbacks

Entry ID: `15542d600c45` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-28 23:06:08 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Lifting (CGL); coarse geometry / large-scale topology of metric spaces (Roe algebras, Property A, Hilbert-space compression of finite-scale lifts) **Field B** (complexity object): Deterministic two-party communication complexity of lifted boolean functions f∘G^n, expressed as the minimum-multiplicity log of a RoeCover over the LiftedInputSpace at scale R_Π

Statement:

> Let G be a MetricGadget for which PropertyAExpand applied to a FolnerWitness produces a certified ε-Hilbert-space embedding with compression exponent α := h(G) > 0 (i.e., the embedding φ satisfies ||φ(u)-φ(v)||₂ ≥ c·d(u,v)^α − C for fixed c,C). Then for every boolean predicate f : {0,1}^n → {0,1} with deterministic decision-tree query complexity Q(f), every deterministic communication protocol Π for f∘G^n yields a CoarsePullbackProtocol whose multiplicity m_Π at scale R_Π = diam(G) satisfies log_2(m_Π) ≥ α · Q(f). Combined with axiom A2 this gives CLC(f, G) ≥ α · Q(f), tightening axiom A4 in regimes where asdim_R(G) is hard to certify but a Følner witness is available.

Rationale (proposer's reasoning):

> This is a direct stress-test of axiom A4 (Property-A Transfer) decoupled from axiom A3 (Asdim Amplification). The coarse embedding produced by PropertyAExpand sends any finite-multiplicity scale-R cover of (X×Y)^n into an □² configuration whose ball-packing number scales like 2^{α·n}. A query-adversary argument on f then pushes the packing requirement up to 2^{α·Q(f)}, which by axiom A2 lower-bounds protocol cost. The conjecture should hold because Property A alone — without quantitative asdim bounds — already supplies the metric distortion needed to beat trivial covering arguments, and it is the minimal coarse hypothesis under which Yu-style descent is known to apply.

Taxonomy category: `LIFTING` (status at proposal time:)

Framework membership: framework `fw_6997a27304`, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f8e94156323e988f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across all 9 (f,G,n) tuples (f∈{AND,OR,PARITY}, G∈{IND_2,EQ_2,IP_2}, n∈{2,3,4}) run with 5 random SDP-init seeds each (45 trials), the conjecture is SUPPORTED iff in ≥80% of trials both c* ≥ ⌊α·Q(f)⌋ and log_2(m_Π) ≥ α·Q(f) − 0.5; FALSIFIED iff any single trial yields c* < ⌊α·Q(f)⌋ − 1 or log_2(m_Π) < α·Q(f) − 1.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Hilbert space compression Property A communication complexity lifting` - `coarse geometry Roe algebra deterministic communication complexity gadget composition` - `Folner sequence compression exponent query-to-communication lifting theorem`

Top relevant hits considered: - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [http://arxiv.org/abs/2601.075 Deep Search for Joint Sources of Gravitational Waves and High-Energy Neutrinos with IceCube During the Third Observing R - [http://arxiv.org/abs/2312.08907v2] Roe algebras of coarse spaces via coarse geometric modules - [http://arxiv.org/abs/2408.07701v2] A categorical interpretation of Morita equivalence for dynamical von Neumann algebras - [http://arxiv.org/abs/1708.01199v6] Coarse quotients by group actions and the maximal Roe algebra - [http://arxiv.org/abs/1508.06106v4] Reversible Denoising and Lifting Based Color Component Transformation for Lossless Image Compression - [http://arxiv.org/abs/2401.04670v1] Modified Levenberg-Marquardt Algorithm For Tensor CP Decomposition in Image Compression

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def hamming_distance(x, y):
    return sum(xi != yi for xi, yi in zip(x, y))

def d_pluss(x, y):
    return sum(hamming_distance(xi, yi) for xi, yi in zip(x, y))
```

```
def sdp_solve(L):
    n = len(L)
    D = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            D[i][j] = (L[i][i] + L[j][j] - L[i][j]) / 2
            D[j][i] = D[i][j]

    # Cholesky decomposition
    A = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1):
            if i == j:
                sum_k = sum(A[k][k] ** 2 for k in range(j))
                A[j][j] = math.sqrt(L[j][j] - sum_k)
            else:
                sum_k = sum(A[k][j] * A[k][i] for k in range(j))
                A[j][i] = (L[j][i] - sum_k) / A[i][i]

    return A

def property_a_expand(X, Y, F):
    n = len(X)
    d_pluss = lambda x, y: sum(hamming_distance(xi, yi) for xi, yi in zip(x, y))
    L = [[d_pluss(X[i], X[j]) for j in range(len(X))] for i in range(len(X))]
    A = sdp_solve(L)

    φ = {}
    for x in X:
        φ[x] = [A[i][X.index(x)] for i in range(n)]

    return φ

def coarse_pullback_protocol(φ, G):
    n = len(G)
    m_Π = 1
    for g in G:
        m_Π *= sum(1 for x in G if all(abs(φ[x][i] - φ[g[i]][i]) < 1e-6 for i in range(n)))
    return m_Π

def run_trial(seed: int) -> dict:
    random.seed(seed)

    gadgets = {
        "IND_2": [[0, 0], [0, 1], [1, 0], [1, 1]],
        "EQ_2": [[0, 0], [0, 1], [1, 0], [1, 1]],
        "IP_2": [[0, 0], [0, 1], [1, 0], [1, 1]]
    }

    predicates = {
        "AND_n": lambda x: all(xi == 1 for xi in x),
        "OR_n": lambda x: any(xi == 1 for xi in x),
        "PARITY_n": lambda x: sum(xi for xi in x) % 2 == 0
    }
```

```
    results = []
    for gadget_name, G in gadgets.items():
        n_values = [2, 3, 4]
        for n in n_values:
            X = list(combinations(range(n), n // 2))
            Y = list(combinations(range(n), n // 2))
            F = [set() for _ in range(3)]
            F[0] = {tuple(sorted(x)) for x in X}
            F[1] = {tuple(sorted(x + y)) for x, y in combinations(X, 2)}
            F[2] = {tuple(sorted(x + y + z)) for x, y, z in combinations(X, 3)}

            φ = property_a_expand(X, Y, F)
            α = sum(math.log(abs(φ[x][i] - φ[y][i])) / math.log(len(G)) for x, y in combinations(X

            f_name = random.choice(list(predicates.keys()))
            f = predicates[f_name]
            Q_f = n

            min_cost = float('inf')
            for c in range(math.ceil(α * Q_f)):
                protocol = []
                for _ in range(c):
                    protocol.append(random.choice([0, 1]))
                if all(f(tuple(sorted(x + y)) == protocol) for x, y in combinations(X, 2)):
                    min_cost = c
                    break

            m_Π = coarse_pullback_protocol(φ, G)

            results.append({
                "metric_name": "slack",
                "metric_value": min_cost - α * Q_f,
                "instances_tested": 1,
                "conjecture_holds": min_cost >= math.floor(α * Q_f) and m_Π >= 2 ** (α * Q_f) / 2,
                "counterexample": ""
            })

    mean_slack = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_slack) ** 2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    return {
        "seed": seed,
        "mean_slack": mean_slack,
        "std_dev": std_dev,
        "support_fraction": support_fraction
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
```

```
        results.append(result)

    mean_slack = sum(r["mean_slack"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["mean_slack"] - mean_slack) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["support_fraction"] >= 0.8) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_slack} std={std_dev} support_fraction={support_fracti
    elif any(r["support_fraction"] < 0.8 for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["support_
        print(f"RESULT: FALSIFIED counterexample=\"not enough support\" first_failing_seed={first_
    else:
        print("RESULT: INCONCLUSIVE insufficient trials")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5de7cf2b.py", line 132, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5de7cf2b.py", line 89, in run_trial
    φ = property_a_expand(X, Y, F)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5de7cf2b.py", line 47, in property_a_ex
    L = [[d_pluss(X[i], X[j]) for j in range(len(X))] for i in range(len(X))]
          ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5de7cf2b.py", line 46, in <lambda>
    d_pluss = lambda x, y: sum(hamming_distance(xi, yi) for xi, yi in zip(x, y))
                           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5de7cf2b.py", line 46, in <genexpr>
    d_pluss = lambda x, y: sum(hamming_distance(xi, yi) for xi, yi in zip(x, y))
                               ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5de7cf2b.py", line 18, in hamming_dista
    return sum(xi != yi for xi, yi in zip(x, y))
                                      ^^^^^^^^^
TypeError: 'int' object is not iterable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a TypeError in hamming_distance before any trial completed, so no data was produced to evaluate the pre-registered support or falsification conditions. | next: Fix the d_pluss/hamming_distance type handling (ensure gadget elements are iterable bitstrings, not ints) and rerun the 45-trial battery.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	7866
2	novelty	claude_max	opus	0	0	3744
3	novelty	claude_max	opus	0	0	8201
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16588
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17231
6	judge	claude_max	opus	0	0	4423

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 58053 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/15542d600c45.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/15542d600c45.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/15542d600c45.tar.gz` (if generated)